Relationer mellan objekt

ArrayList eller array för samlingar av objekt? Använd en array om antalet objekt i samlingen alltid är detsamma, i annat fall ArrayList.

I följande uppgifter att det är viktigt att, där så anges, metoder inte lämnar ut referenser till privata objekt, arrayer eller listor. För att vara säker på att du implementerat detta korrekt räcker det inte att få samma utskrift som i testkoden, du måste även studera implementationen i lösningsförslagen.

Notera att problemet ovan inte gäller primitivt data och objekt som är ”immutable”.

1. Delat aggregat, shared aggregation

Hämta klasserna Account och ListOfAccounts från föreläsningen.

Uppdatera klassen ListOfAccounts så att den istället för en Account-array använder använder en lista, ArrayList<Account>, internt. Notera att du inte ska ändra något i klassens publika gränssnitt (de publika metodernas signaturer) utan endast i den interna implementationen. Det ska alltså vara möjligt att exekvera samma main-metod som tidigare.

Att tänka på:

- Du kan inte indexera med ”[…]” i en ArrayList. Anänd i stället
  - list.get(index) för att avläsa ett värde
  - list.add(…) för att lägga till ett objekt sist i listan
  - list.remove(index) för att ta bort ett objekt i listan
- Du behöver inte längre datamedlemmen nOfAccs eftersom listan håller reda på aktuella antalet element. Använd metoden size när du behöver veta antalet element.
- Du behöver inte egna metoder för att expandera/utöka listan respektive packa listan. Det sköts internt av list-klassens metoder.

Eftersom klassens publika gränssnitt inte ändras i denna uppgift, endast den interna implementationen, behöver inte main-metoden med test-kod uppdateras.

2. Kortspel

En kortlek

Skriv en klass som representerar en kortlek, Deck. Använd en privat lista, ArrayList<Card>, för att hantera korten internt i leken. Använd klassen Card samt enum Rank och Suit, som finns bland föreläsningsmaterialet (och i lösningsförslaget).

Följande funktionalitet ska finnas:

- Konstruktorn skapar listan samt de 52 unika korten.[1]

[1] För att iterera över alla värden i en enum kan du använda metoden values, till exempel:
Suit[] suites = Suit.values(); - ger en array med alla värden i denna enum.

- Metoden fill används för att fylla listan, och alltså kortleken, med 52 nya kort (glöm inte att tömma leken på ev. kvarvarande kort först). Använd metoden fill i konstruktorn.
- Metoden dealCard lämnar ut en referens till det översta kortet i leken samt tar bort detta från den interna listan. Detta ska vara enda sättet att ”komma åt” kort i leken.
- Metoden shuffleCards blandar korten i leken. Notera att blandningen ska fungera korrekt även när leken inte är fylld. Klassen java.util.Collections har klass-metoder för att blanda objekten i en lista (sök efter metoden shuffle i API-dokumentationen för Collections).

Skriv också en main-metod som testar kortlekens funktionalitet.

En hög med kort

VI behöver representera kort som delats ut till exempelvis spelares händer eller till en hög på spelbordet. Vi skulle kunna använda oss av publika list-objekt för detta, men det finns situationer då korten i handen/högen inte får manipuleras hur som helst, så vi skriver en specifik klass för detta, Pile. Pile har internt en privat lista med kort.

Skriv klassen Pile med nedanstående metoder.

- En konstruktor som skapar en tom lista för kort.
- getSize – returnerar antalet kort
- clear – tömmer korthögen
- add(Card) -lägger till ett kort sist i listan
- get(int) – returnerar en referens till kortet på angiven position, utan att ta bort det
- remove(int) - returnerar en referens till kortet på angiven position och tar bort kortet från listan
- en egen version av toString

Studera API-dokumentationen för klassen ArrayList – många av metoderna ovan har en motsvarighet i list-klassen.

Skriv slutligen en main-metod som testar kortlekens funktionalitet.

3. Auktion

Skriv en klass som representerar en användare. En användare har ett användarnamn, e-postadress samt ett lösenord. Det ska gå inte gå att ändra namn och e-post när ett objekt väl skapats, däremot ska det gå att ändra lösenordet.

Skriv sedan en klass som representerar en auktion och en klass som representerar ett bud. En auktion har en användare, som äger auktionen, en vara (får representeras av en textsträng) samt ett antal bud. Det ska gå att lägga till bud på en auktion, samt hämta en kopia av listan med buden.

Ett bud har en användare, budgivaren, samt ett belopp som data. Ett bud ska inte kunna ändras efter det skapats (objekt av denna typ ska vara ”immutable”).

Skriv också ett main där du skapar användare, en auktion samt lägger till ett antal bud för denna auktion.

Testkod:

```
User anders = new User("Anders", "aslm@kth.se", "42");
User reine = new User("Reine", "rebm@kth.se", "4711");

Auction auction = new Auction(reine, "The Shining, DVD");
auction.addBid(new Bid(anders, 100));
auction.addBid(new Bid(anders, 200));

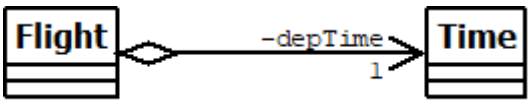
System.out.println(auction.toString());

Output:
Reine, The Shining, DVD [bids: Anders: 100; Anders: 200; ]
```

4. Komposition, composite aggregation

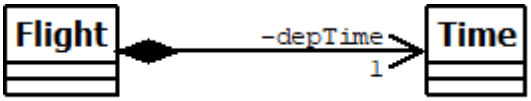
a) Skriv en enkel klass Time som representerar ett klockslag med h, m och s. Det ska finnas en metod för att ändra tiden, `setTime(int h, int m, int s)`.
Skriv sedan en klass Flight som representerar en avgång på en flygplats. Datamedlemmar är, destination, fligthnr och departureTime (avgångstid), det senare ett Time-objekt. Det ska finnas en metod, getDeparture, som returnerar avgångstiden som ett objekt.

Denna lösning är ett exempel på ”shared agregation”, andra klasser kan komma åt objektet depTime eftersom getDeparture returnerar en kopia av referensen, inte en kopia av objektet.



b) Ändra klassen Flight så att
- konstruktorn skapar det inkapslade objektet departureTime
- metoden getDeparture så att den returnerar en kopia av tidobjektet istället för en referens (så att det inte går att ändra i den interna avgångstiden via den referens som returneras).

Denna variant på lösningen är ett exempel på ”composite aggregation” – eftersom referenser till det privata objektet aldrig lämnas ut.



Testkod, se nedan – notera att fall a och b ger olika utfall när den tid som fås via getDepTime ändras.

```
FlightA flight = new FlightA("Stockholm", "SK1234", new Time(12, 30));
System.out.println(flight.toString());

Time depTime = flight.getDepTime();
depTime.tic(1800); // seconds
System.out.println(flight.toString());

Output A:
Stockholm, SK1234, dep time: 12:30:00
Stockholm, SK1234, dep time: 13:00:00 // !

Output B (konstruktoranropet ovan behöver modifieras):
Stockholm, SK1234, dep time: 12:30:00
```

```
Stockholm, SK1234, dep time: 12:30:00 // getDepTime returns a copy
```

5. Uthyrning

Örnsbergs Kanotklubb behöver hjälp med utlåning av sina kajaker. För detta behövs klasser som representerar medlemmar, kajaker, lån samt en klass, LoanManager som hanterar alla kajaker och lån.

Du ser i kodfragmenten nedan vilka datamedlemmar som ska finnas i respektive klass. Inga andra datamedlemmar ska deklareras.
Alla klasser ska, förutom det som nämns specifikt nedan, ha konstruktorer som initierar *alla* datamedlemmar korrekt, get-metoder, set-metoder där det är lämpligt och en omdefiniering av toString.

```
public class Member {
    private final String fullName;
    private final Pnr pnr;
    …
```

Klassen Pnr representerar ett personnummer. Denna klass ska *inte* ha någon public konstruktor. Istället ska det finnas en statisk metod (klassmetod) som givet en textsträng som representerar ett personnummer kontrollerar om detta är korrekt; om så är fallet returneras ett nytt objekt, i annat fall kastas ett IllegalArgumentException. Kontrollen görs med `pnrStr.matches(pnrPattern)`.

```
public class Pnr {
    private final String pnrStr;
    private static final String pnrPattern =
        "^(19|20)?[0-9]{6}[- ]?[0-9]{4}$"
    …
```

En kajak har, förutom en modell, nivå och nummer, en medlem som ansvarar för kajaken (careTaker); konstruktorn måste sätta en medlem som ansvarig, men denne kan bytas ut senare.

```
public class Kayak {
    private final String model;
    private final KLevel level;
    private final long number;
    private Member caretaker;
    …
```

Level är en enum som kan ha något av värdena BEGINNER, EASY, SKILLED eller ADVANCED.

Objekt som representerar ett lån ska vara ”immutable”.

```
public class Loan {
    private final Kayak kayak;
    private final Member borrower;
    private final LocalDate loanDate;
```

Klassen LoanManager ska hålla en lista med alla klubbens kajaker samt en lista med alla lån. Inga fler datamedlemmar ska finnas. Klassens funktionalitet definieras av interfacet ILoanManager, se nedan.

Din klass ska implementera detta interface och får inte ha några andra publika metoder förutom konstruktorn.

```
public interface ILoanManager {
    List<Kayak> getKayaks();
    List<Loan> getLoans();
    void addKayak(Kayak kayak);
    void addLoan(Member borrower, Kayak kayak, LocalDate loanDate)
       throws IllegalStateException;
    List<Loan> getLoansFor(Kayak kayak)
    List<Kayak> getAvailableKayaks(LocalDate targetDate);
}
```

Krav på metoderna:

- getKayaks och getLoans ska returnera en verklig kopia av respektive lista.
- addkAyak ska bara lägga till en ny kajak om den inte redan existerar i listan.
- addLoan ska lägga till ett nytt lån i listan, men om vald kajak inte är upptagen valt datum ska istället ett undantag kastas.
- getLoansFor ska returnera en lista med alla lån för aktuell kajak.
- getAvailableKayaks ska returnera en lista med alla kajaker som inte är upptagna (inte ingår i ett lån) det aktuella datumet (targetDate).

Testkod (ej komplett):

```
Pnr andersPnr = Pnr.createPnr("600101-1111"); // constructor is private
Member anders = new Member("Anders Lindström", andersPnr);
System.out.println(anders.toString());

Kayak k1 = new Kayak("WKV 540", KLevel.SKILLED, 1, anders);
Kayak k2 = new Kayak("Point 65", KLevel.EASY, 2, anders);

LoanManager manager = new LoanManager();
manager.addKayak(k1);
manager.addKayak(k2);
System.out.println(manager.toString());

List<Kayak> available = manager.getAvailableKayaks(LocalDate.now());
System.out.println("Available today: " + available.toString());

manager.addLoan(anders, k1, LocalDate.now());
available = manager.getAvailableKayaks(LocalDate.now());
System.out.println("Available today: " + available.toString());

// this will throw an exception
manager.addLoan(anders, k1, LocalDate.now());
```

Output:

```
Member {fullName='Anders Lindström', ssn: 600101-1111}

LoanManager{kayaks: 2, loans: 0}

Available today: [Kayak{model='WKV 540', level=SKILLED, number=1},
Kayak{model='Point 65', level=EASY, number=2}]
```

```
Available today: [Kayak{model='Point 65', level=EASY, number=2}]

Exception in thread "main" relations.rental.IllegalLoanException: WKV 540,
2021-09-07
```